

웹 데이터 수집 개요

이 경 미

웹 데이터 수집 원리 이해

웹 데이터 수집 원리 이해

■ 학습 목표

- Python 크롤링 코드를 작성하기 전에 웹에서 데이터가 어떻게 전달되고, 크롤러가 그 데이터를 어떻게 가져오는지를 이해

■ 학습 내용

- 웹의 Client-Server 구조
- URL의 구조와 역할
- HTTP Request와 Response의 동작 과정
- HTML과 DOM의 관계
- 정적 웹페이지와 동적 웹페이지의 차이
- Requests, BeautifulSoup, Selenium의 역할 차이
- 웹페이지에 보이는 데이터가 어디에서 전달되는지 이해를 위한 개발자 도구 활용

웹 크롤링이란?

- 프로그램을 이용하여 웹페이지에 자동으로 접근하고 필요한 정보를 수집하는 기술

일반 사용자의 웹 이용 (브라우저)



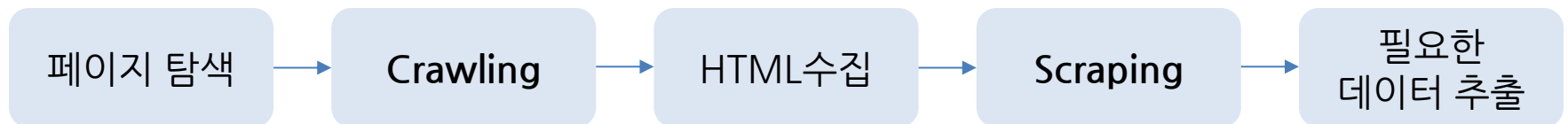
크롤링 프로그램의 동작 (자동화)



Crawling과 Scraping

- 웹 데이터 수집 프로그램에서 두 과정이 함께 사용되는 경우가 많음

구분	Crawling	Scraping
목적	웹페이지 탐색/수집	특정 데이터 추출
주요 대상	페이지, URL	제목, 가격, 날짜 등
특징	여러 페이지 탐색	필요한 데이터 선택
예	뉴스 페이지 순회	뉴스 제목 추출



웹의 기본 목적과 구성

■ 웹의 기본 목적

- 다른 여러 컴퓨터에서 문서를 공유하거나 보는 목적
- 웹에서 다루는 문서를 웹 문서라고 부름

■ 웹의 구조

- 인터넷을 활용하여 거미줄처럼 연결된 정보 소통 망
- World Wide Web
 - 웹 문서를 인터넷 상의 컴퓨터들끼리 주고 받는 네트워크 시스템

■ 웹의 구성

- 웹 서버와 웹 클라이언트 컴퓨터들로 구성

인터넷과 웹은 다르다

■ 인터넷(Internet)

- 컴퓨터 연결 네트워크
 - 1969년 미 국방성 고등 연구 계획국(ARPA)
 - 여러 대학들과 계약 업체 사이의 컴퓨터 연결
- 컴퓨터마다 고유한 주소(IP)를 부여하여 컴퓨터 구분
- 인터넷을 활용하는 응용 서비스 사례
 - 전자우편(e-mail) / 뉴스(news) / 파일 전송(ftp)
 - 채팅(Internet Relay Chat) / 메신저(Messenger)
 - P2P(Peer to Peer) / 스트리밍 서비스(Streaming Service)
 - 인터넷 전화기(Internet Phone) /월드 와이드 웹(World Wide Web)

■ 웹(WWW)

- 인터넷을 활용하는 응용 서비스 중의 하나
- 웹 서버와 웹 브라우저로 구성되는 정보 전달 및 공유 서비스

■ 인터넷이 고속도로라면 웹은 고속도로 망을 이용한 물류 산업

웹의 시작

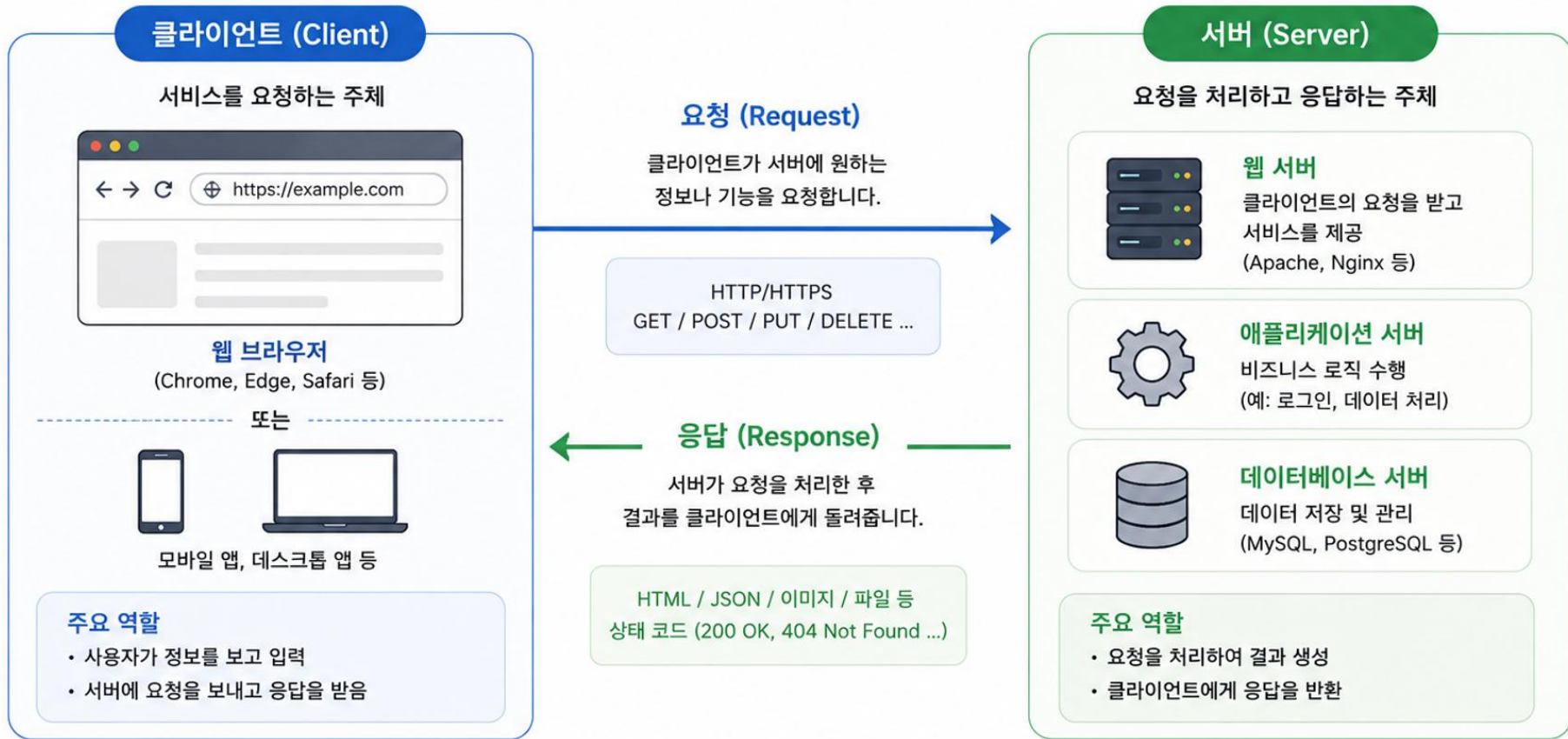
- Tim Berners-Lee의 아이디어에서 시작
 - 1989년 웹 개념 제안
 - 1990년 WorldWideWeb 프로젝트 시작
 - 서버-클라이언트로 동작하는 HTTP 모델 창시
 - HTML 언어 개발, 하이퍼링크 개념 구현
 - 세계 최초의 웹 서버와 웹 브라우저 개발



- Tim Berners-Lee가 CERN에서 사용한 NeXT 컴퓨터
- 최초로 개발한 웹 서버 소프트웨어를 설치하고 운용
- 웹 브라우저 WorldWideWeb를 개발하는데 이용
(참조 : https://en.wikipedia.org/wiki/World_Wide_Web)

웹의 기본 구조

■ 서버와 클라이언트



웹 동작 과정

■ 요청과 응답 과정

■ 요청(request)

- 웹을 통해 서비스/리소스를 요구하는 것
- 요청을 하는 주체: 클라이언트(client)

■ 응답(response)

- 클라이언트의 요청에 대해 응답을 보내주는 것/서비스를 제공하는 것
- 응답을 하는 주체: 서버(server)

웹 동작 과정

요청 (Request) 과정

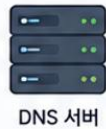
1 URL 입력

사용자가 브라우저 주소창에 URL 입력



2 DNS 조회

도메인(example.com)에 해당하는 서버의 IP 주소를 DNS 서버에 요청



example.com
IP 주소는?

3 TCP 연결

서버(IP 주소)와 통신하기 위해 네트워크 연결(TCP) 수립



4 HTTP 요청

브라우저가 서버에 페이지 요청을 보냄

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Chrome/...
Accept: text/html
...
```

인터넷



웹 서버
(example.com)
IP 주소 203.0.113.10

응답 (Response) 과정

8 웹 페이지 표시

최종적으로 사용자가 웹 페이지를 확인



7 HTML 파싱

HTML, CSS, JavaScript 등을 해석하여 화면을 구성



6 브라우저 수신

브라우저가 응답을 받아 내용을 처리



5 HTTP 응답

서버가 요청을 처리하고 결과(HTML 등)를 응답으로 보냄

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1256

<html> ... </html>
```

요청 (Request)

정보를 달라고 요청하는 과정



인터넷



웹 서버

데이터 이동
(요청 → 응답)



웹 서버



인터넷



사용자
(브라우저)

응답 (Response)

요청에 대한 결과를 돌려주는 과정

URL 이해하기

■ URL(Uniform Resource Locator)

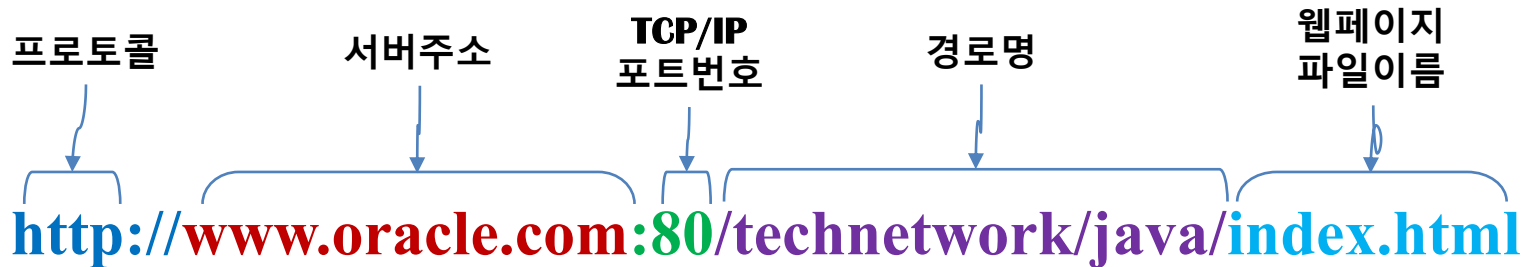
- 자원 위치 표시자
- 인터넷에서 자원의 위치를 나타내는 주소
- resource: 문서, 동영상, 사진 등을 자원이라고 부름
- 인터넷 상에 퍼져 있는 자원들의 위치를 찾아낼 수 있는 주소

■ 도메인 주소

- 예: `www.naver.com`
- 네트워크 상의 컴퓨터에 대한 주소는 숫자와 점으로 이루어진 IP 주소 사용 => 읽고 기억하기 쉽도록 도메인 주소를 사용
- 도메인 네임 서버: 도메인 주소 → IP 주소

URL 이해하기

■ URL(Uniform Resource Locator)

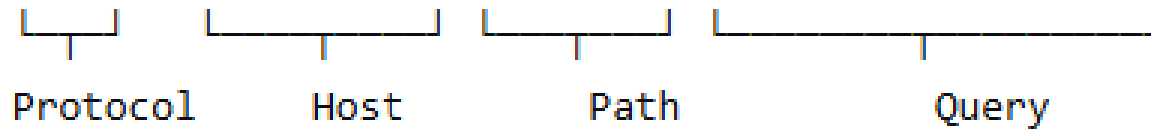


- **프로토콜** : HTTP, https, file, ftp, telnet, mailto, news 등
- **서버주소** : 웹 페이지를 가진 컴퓨터의 인터넷 주소, IP 주소
- **TCP/IP 포트 번호** : 서버가 브라우저로부터 접속을 기다리는 TCP/IP 포트 번호.
프로토콜마다 다르며, HTTP의 경우 80, telnet은 23
- **경로명** : 웹 서버 내 웹 페이지 파일의 폴더 경로
- **파일이름** : 웹 페이지의 HTML 파일 이름

URL 이해하기

■ URL(Uniform Resource Locator)

`https://example.com/products?page=2&category=book`

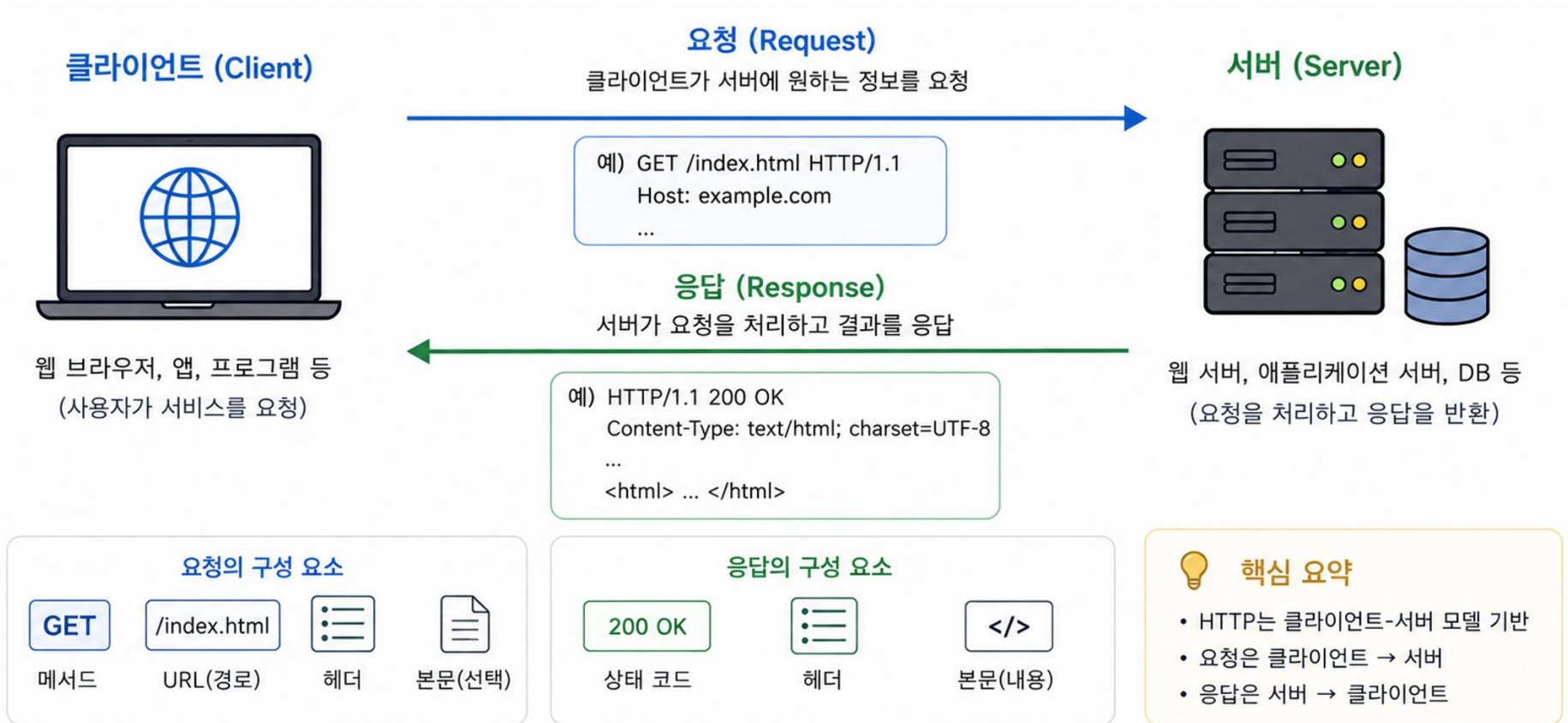


구성요소	예	의미
Scheme	<code>https</code>	통신 방식
Host	<code>example.com</code>	서버 주소
Path	<code>/products</code>	요청할 자원
Query	<code>page=2</code>	서버에 전달할 조건

HTTP란?

■ HTTP(HyperText Transfer Protocol)

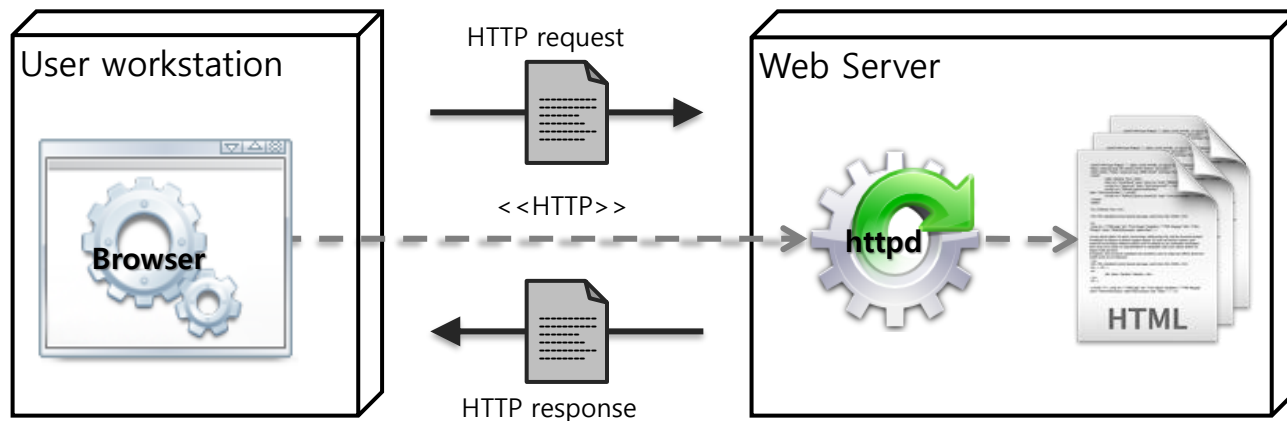
- 웹에서 데이터를 주고받기 위한 통신 규칙



HTTP 요청과 응답

■ HTTP Request와 HTTP Response

- 브라우저가 서버의 페이지를 요청하면 서버는 해당 파일을 찾은 다음 HTTP응답을 통해 클라이언트에 전송
- 브라우저는 응답된 페이지를 해석하여 화면에 보여줌
- Request
 - 사용자가 원하는 파일 또는 리소스의 위치와 브라우저에 관한 정보를 포함
- response
 - 요청한 자원에 관한 정보를 가지고 있으며, 일반적으로 텍스트 형태이며, 그래픽 등을 바이너리 정보를 포함할 수 도 있음.

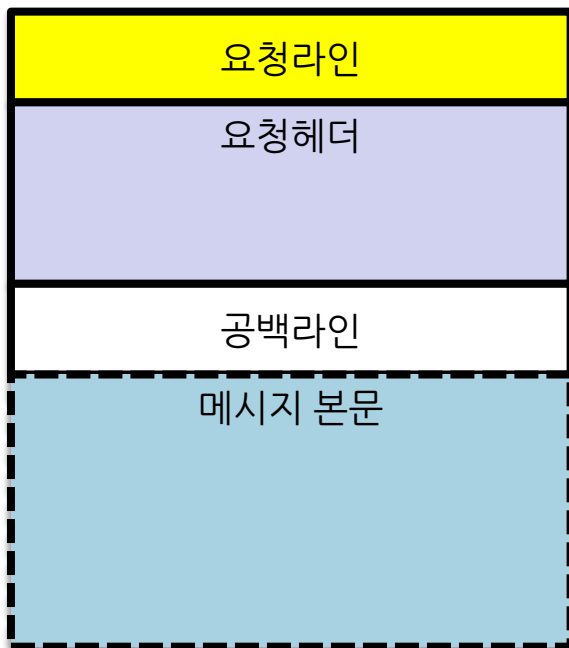


HTTP 요청과 응답

■ HTTP Message

- HTTP Message는 크게 요청과 응답 메시지로 구분
- HTTP Message는 크게 라인, 헤더, 바디 영역으로 나뉘며, 또 여기서 헤더와 바디 영역은 공백으로 구분됨
- 바디 영역을 제외한 나머지 영역(요청, 헤더)의 각 라인은 연속된 CR/LF(\r\n) 문자로 구분

HTTP Request 메시지의 구성



HTTP Response 메시지의 구성



HTTP 요청과 응답

■ HTTP Header

- HTTP 요청과 응답에 대한 부가 정보 전달
- 대표적인 요청 Header

헤더	의미	설명	예
User-Agent	클라이언트 정보	요청을 보내는 브라우저·프로그램·OS 등의 정보를 전달	Mozilla/5.0 ... Chrome/...
Accept	받을 수 있는 데이터 형식	클라이언트가 처리할 수 있는 콘텐츠 형식을 서버에 전달	text/html, application/json
Accept-Language	선호 언어	클라이언트가 원하는 응답 언어를 서버에 전달	ko-KR,ko;q=0.9
Referer	이전 페이지 주소	현재 페이지로 이동하기 전에 방문했던 페이지의 URL을 전달	https://example.com/list
Cookie	사용자 상태 정보	서버가 이전에 저장하도록 한 쿠키를 다시 서버에 전달	sessionid=abc123

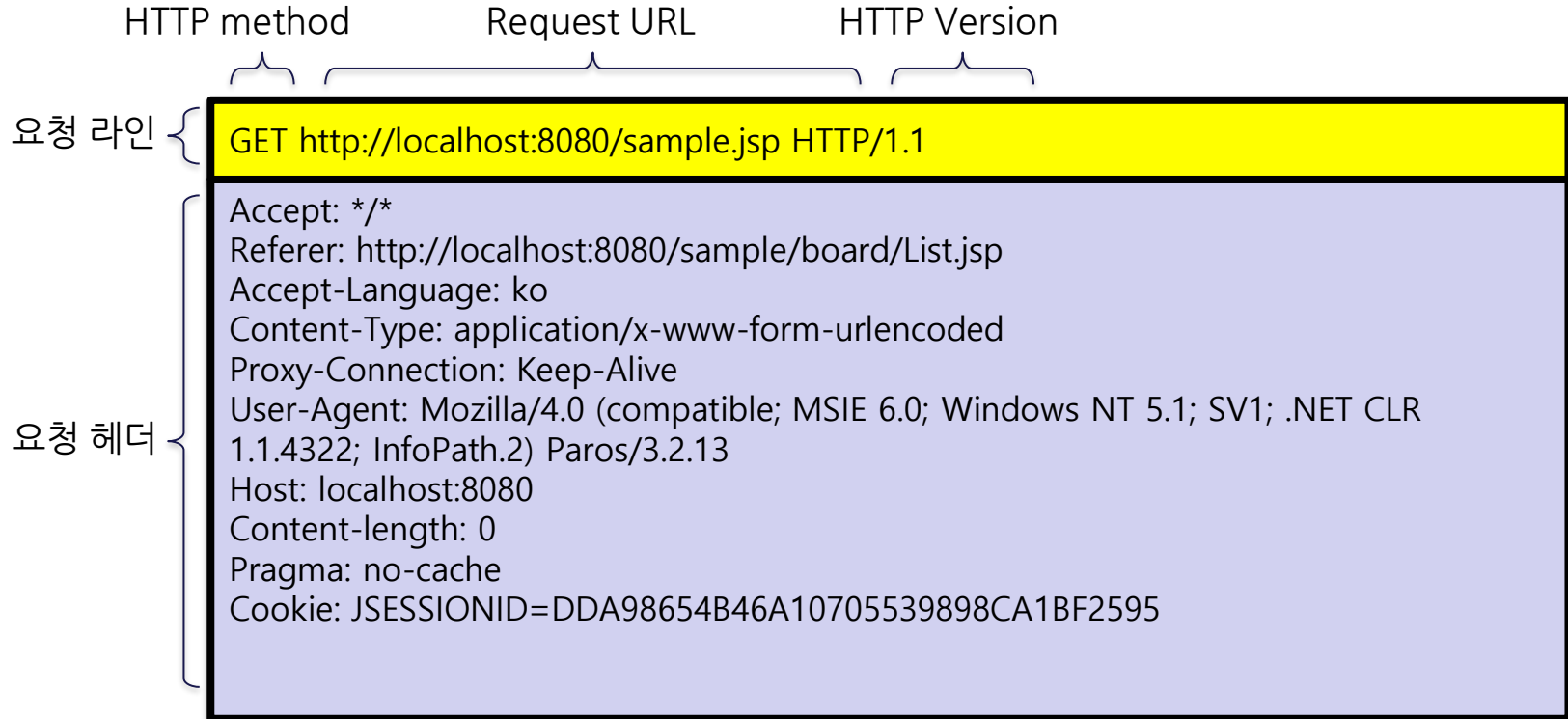
HTTP 요청과 응답

■ HTTP 요청 방식

요청 방식	설명
OPTIONS	서버 자원 및 기능검색을 위한 요청 보통 Web 서버가 어떤 Method를 지원하는지 알기 위해 사용
GET	요청하는 주소(URL)에 의해 서버의 자원을 요청. 서버에 전달되는 해당 데이터(파라미터)가 요청 URL에 포함
HEAD	바디 영역을 제외한 서버 정보를 요청 요청하는 URI의 자원 유/무를 알아내기 위해 사용
POST	GET 요청과는 달리 해당 데이터(파라미터)가 요청 URL에 포함되지 않으며 요청 바디 영역에 포함
PUT	바디 영역의 내용을 실제 서버 URI 위치에 파일로 저장
DELETE	서버에 저장된 자원을 삭제
TRACE	요청 자원이 수신되는 경로를 보여줌
CONNECT	프록시와 같은 중간 서버에 터널을 형성하여 요청하기 위해 사용

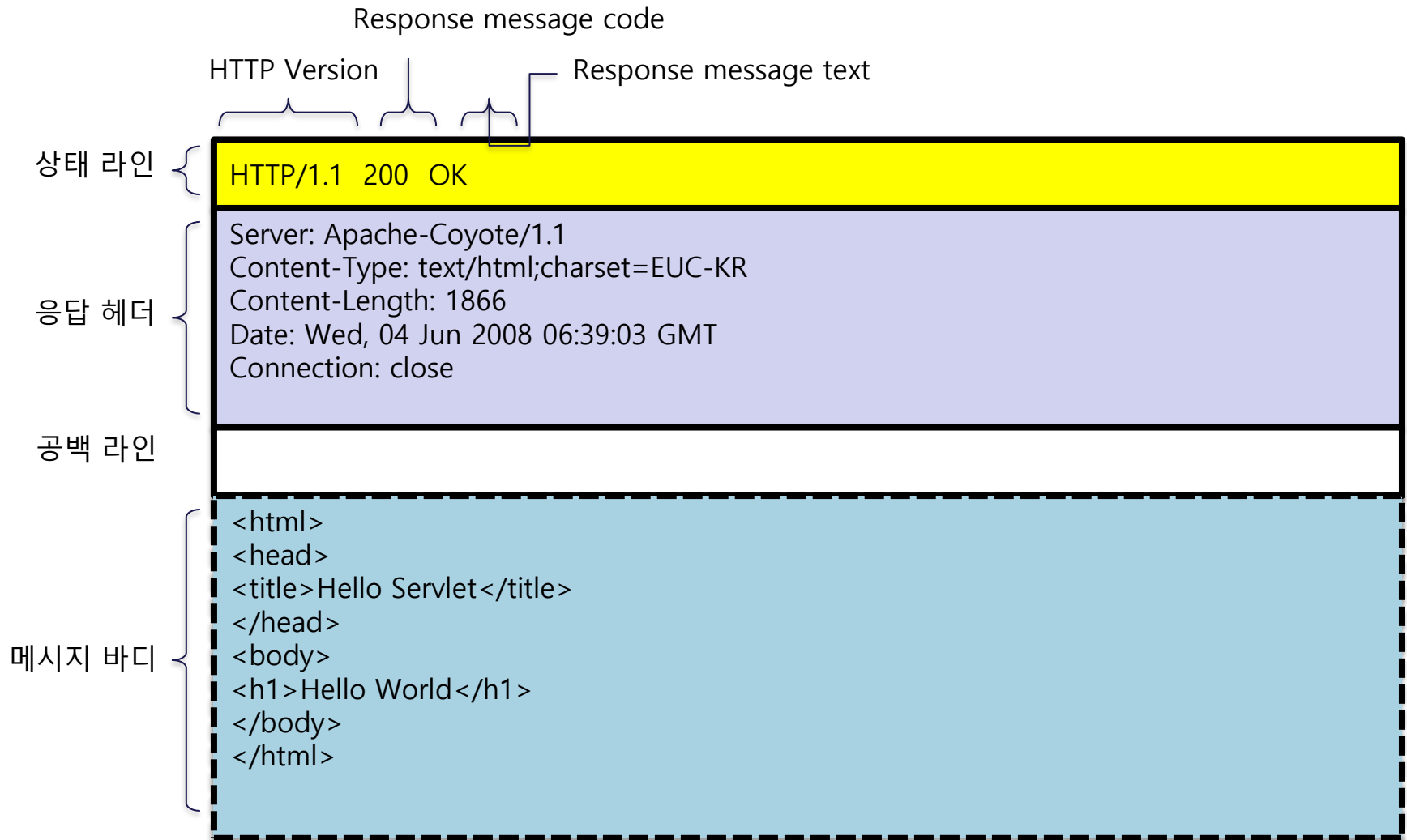
HTTP 요청과 응답

■ HTTP GET Request 메시지



HTTP 요청과 응답

■ HTTP Response 메시지



HTTP 요청과 응답

■ HTTP Status Code

상태 코드	의미	내용
100번 영역	정보전송	임시적인 응답을 나타내는 것은 Status-Line과 선택적인 헤더들로 구성되어 있고, 빈 줄로 끝을 맺는다.
200번 영역	성공	클라이언트의 요구가 성공적으로 수신되어 처리되었음을 의미 200(성공), 201(POST요청 처리), 204(전송할 데이터 없음)
300번 영역	리다이렉션	해당 요구사항을 처리하기 위해서는 사용자 에이전트에 의해 수행되어야 할 추가적인 동작이 있음을 의미 (다른 URL로 이동)
400번 영역	클라이언트 측 오류	클라이언트가 서버에게 보내는 요구 메시지를 완전히 처리하지 못한 경우와 같이 클라이언트에서 오류가 발생한 경우에 사용 401(사용자인증), 403(접근권한 없음), 404(요청한 URL 없음) 429(너무 많은 요청)
500번 영역	서버측 오류	서버 자체에서 발생한 오류상황이나 요구사항을 제대로 처리할 수 없을 때 사용. 500(서버오류), 503(서비스 사용 불가)

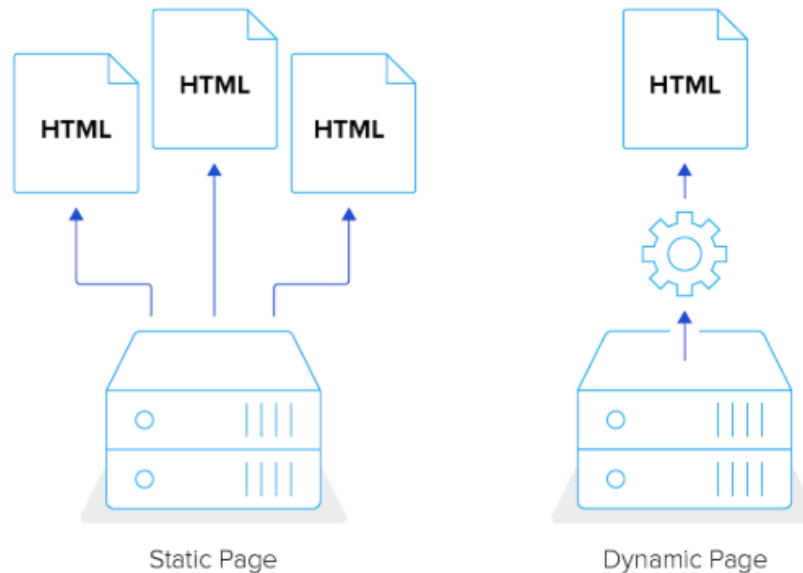
정적/동적 웹페이지

■ 정적 웹페이지

- 서버에 저장된 웹페이지를 그대로 클라이언트로 전달
- 서버에 저장된 리소스(html, image, js, css 등)가 그대로 전달됨

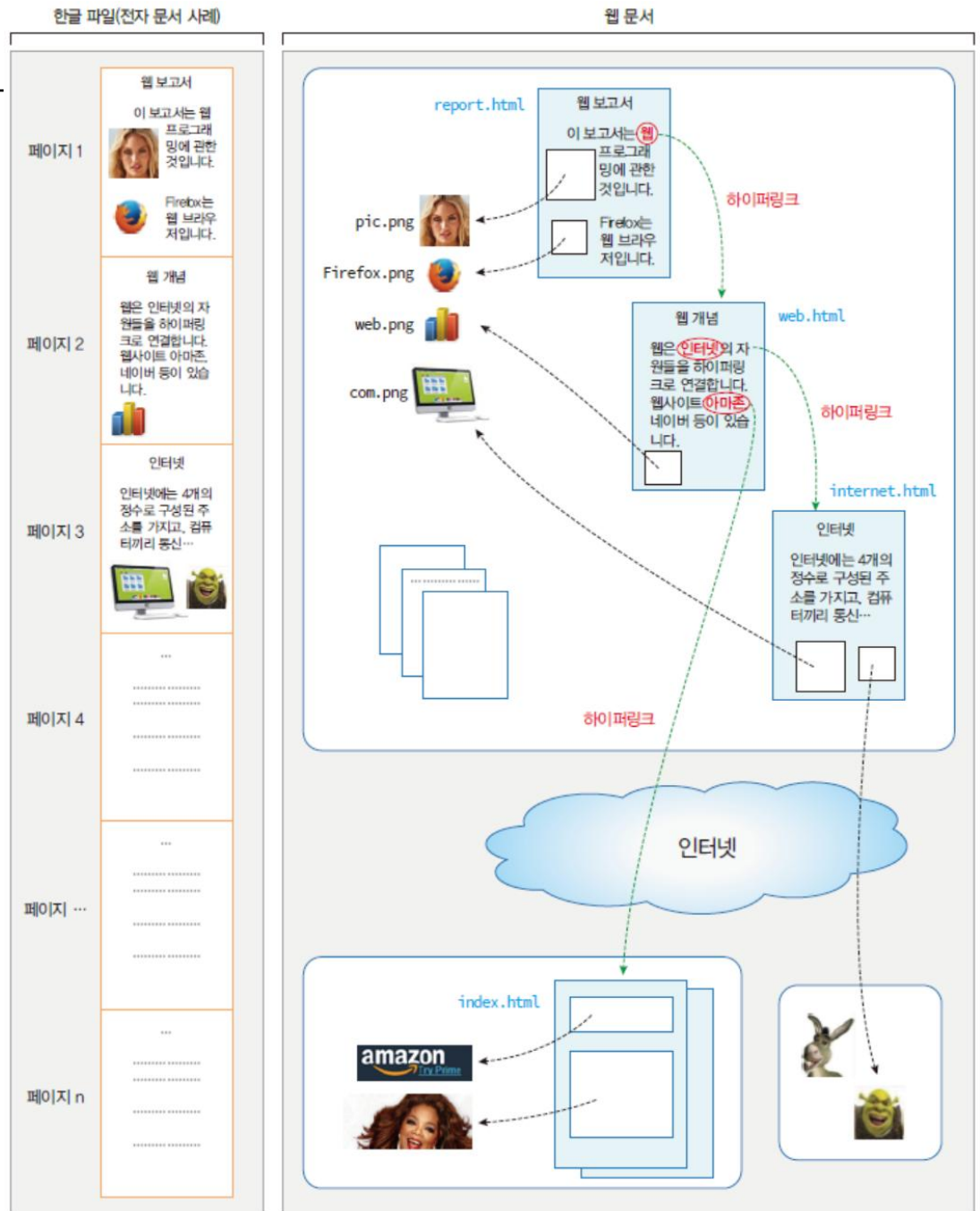
■ 동적 웹페이지

- 서버는 사용자의 요청을 해석한 후 가공된 페이지를 전달



이미지출처: -<https://www.toptal.com/front-end/static-site-generators-comparison-2018>

웹 문서와 전자 문서



웹 페이지 만들기

■ 웹 페이지 구성 3 요소

- 웹 페이지의 구조와 내용 - HTML (HyperText Markup Language)
- 웹 페이지의 모양 - CSS(Cascading Style Sheet)
- 웹 페이지의 행동 및 응용 프로그램 - Javascript

■ 웹 페이지는 3 요소를 분리하여 개발

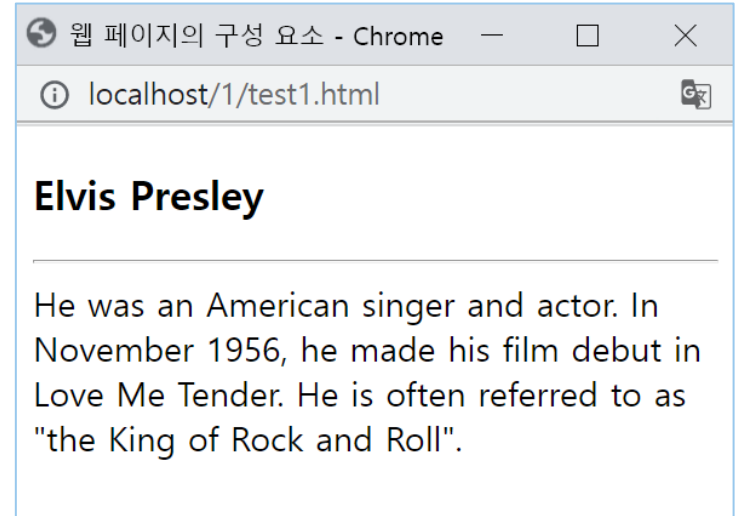
웹 표준 기술



웹 페이지 만들기

■ HTML 태그로 문서의 구조와 내용 만들기

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>웹 페이지의 구성 요소</title>
</head>
<body>
<h3>Elvis Presley</h3>
<hr>
He was an American singer and actor. In November
1956, he made his film debut in <span>Love Me
Tender</span>. He is often referred to as
"<span>the King of Rock and Roll</span>".
</body>
</html>
```



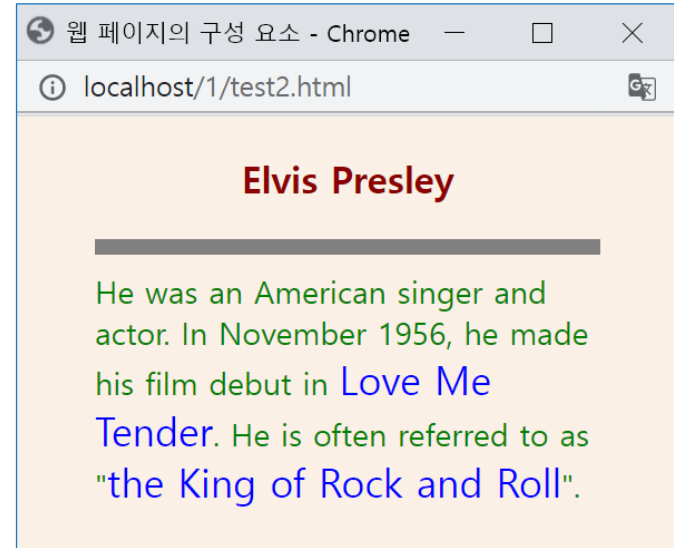
HTML 태그로 구조와 내용만 있는 웹 페이지

웹 페이지 만들기

■ CSS로 문서 모양 만들기

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>웹 페이지의 구성 요소</title>
<style>
  body { background-color : linen; color : green;
        margin-left : 40px; margin-right : 40px;}
  h3 { text-align : center; color : darkred;}
  hr { height : 5px; border : solid grey;
      background-color : grey }
  span { color: blue; font-size: 20px; }
</style>
</head>
<body>
<h3>Elvis Presley</h3>
<hr>
He was an American singer and actor. In November
1956, he made his film debut in <span>Love Me
Tender</span>. He is often referred to as
"<span>the King of Rock and Roll</span>".
</body>
</html>
```

CSS로 문서의 모양(스타일) 코딩



웹 페이지 만들기

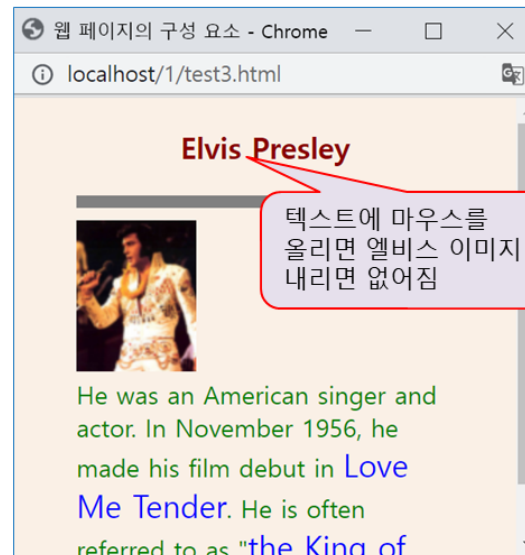
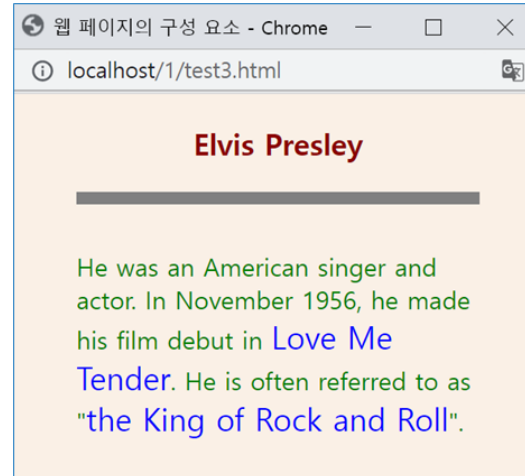
■ Javascript로 사용자 인터페이스 처리

```
<!DOCTYPE html>
<html>
<head><meta charset="utf-8">
<title>웹 페이지의 구성 요소</title>
<style>
  body { background-color : linen; color : green;
        margin-left : 40px; margin-right : 40px;}
  h3 { text-align : center; color : darkred;}
  hr { height : 5px; border : solid grey;
      background-color : grey }
  span { color: blue; font-size: 20px; }
</style>
<script>
  function show() { // <img>에 이미지 달기
    document.getElementById("fig").src = "ElvisPresley.png"
  }
  function hide() { // <img>에 이미지 제거
    document.getElementById("fig").src= "";
  }
</script>
</head>
<body>
<h3 onmouseover="show()" onmouseout="hide()">
  Elvis Presley</h3>
<hr>
<div><img id="fig" src=""></div>
He was an American singer and actor. In November
1956, he made his film debut in <span>Love Me
Tender</span>. He is often referred to as
"<span>the King of Rock and Roll</span>".
</body>
</html>
```

자바스크립트 코드 추가

텍스트에 마우스를 올리면 show() 함수 호출

텍스트에 마우스를 올리면 엘비스 이미지 출력. 내리면 없어짐

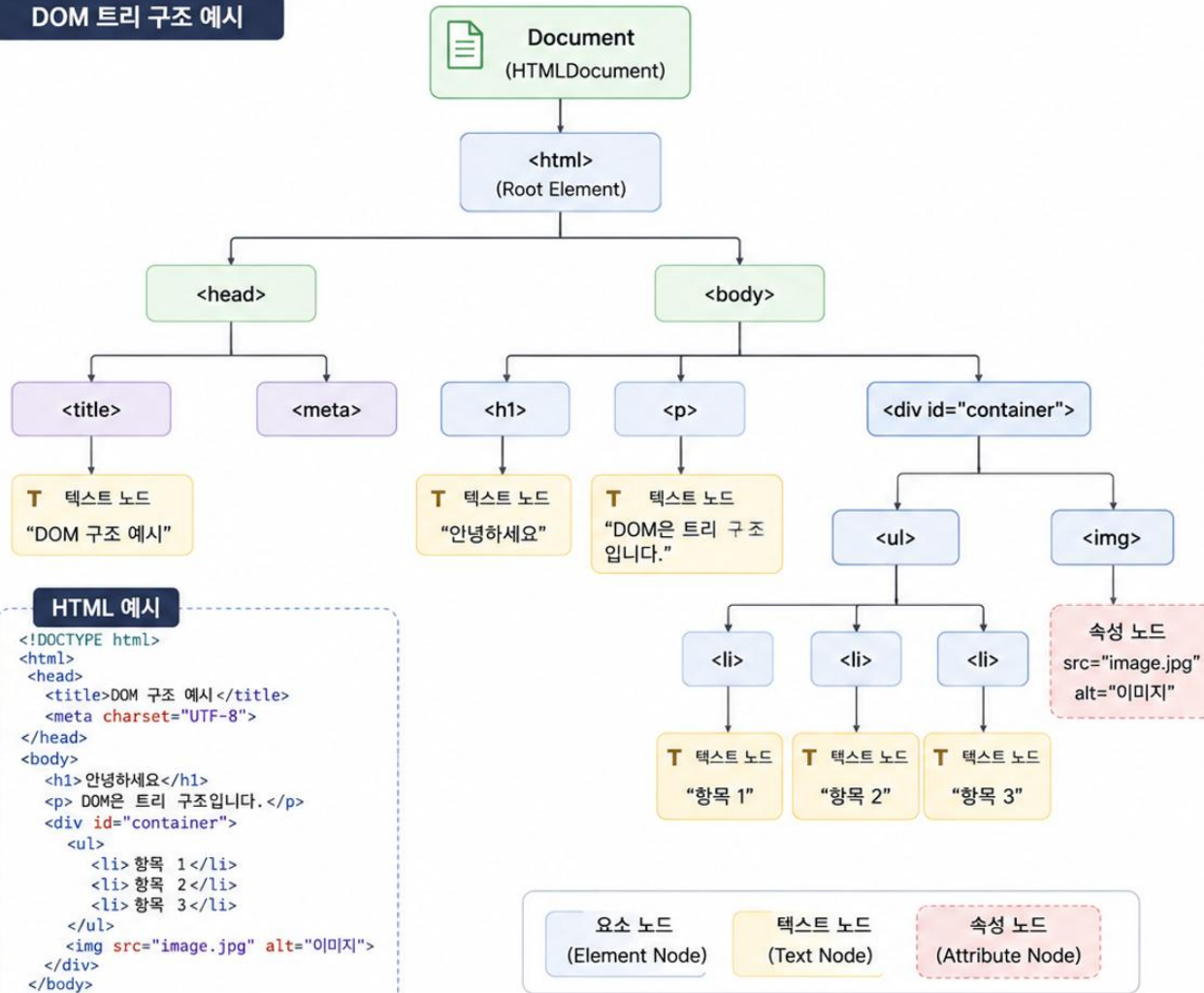


- Document Object Model, 문서 객체 모델

- HTML 문서를 프로그램이 다룰 수 있도록 트리 구조로 표현한 모델
- HTML 문서를 파싱해서 만들어진 객체
- HTML 문서에서 원하는 요소를 찾기 위해서는 요소를 찾는 메소드를 실행시키기 전에 먼저 DOM이 만들어져 있어야 함
- HTML 문서는 태그, 속성, 텍스트 등이 트리 구조로 표현됨

DOM

DOM 트리 구조 예시



DOM 구성 요소 설명



Document 노드

- DOM 트리의 최상위 노드
- 문서 전체를 대표



요소(Element) 노드

- HTML 태그를 나타냄
- 자식 노드(요소/텍스트/속성 등을 가짐)



텍스트(Text) 노드

- 태그 내부의 텍스트 내용
- 요소 노드의 자식 노드



속성(Attribute) 노드

- 요소의 속성(이름=값)
- 요소 노드에 연결



주석(Comment) 노드

- HTML 주석
- 트리 구조에 포함될 수 있음

특징

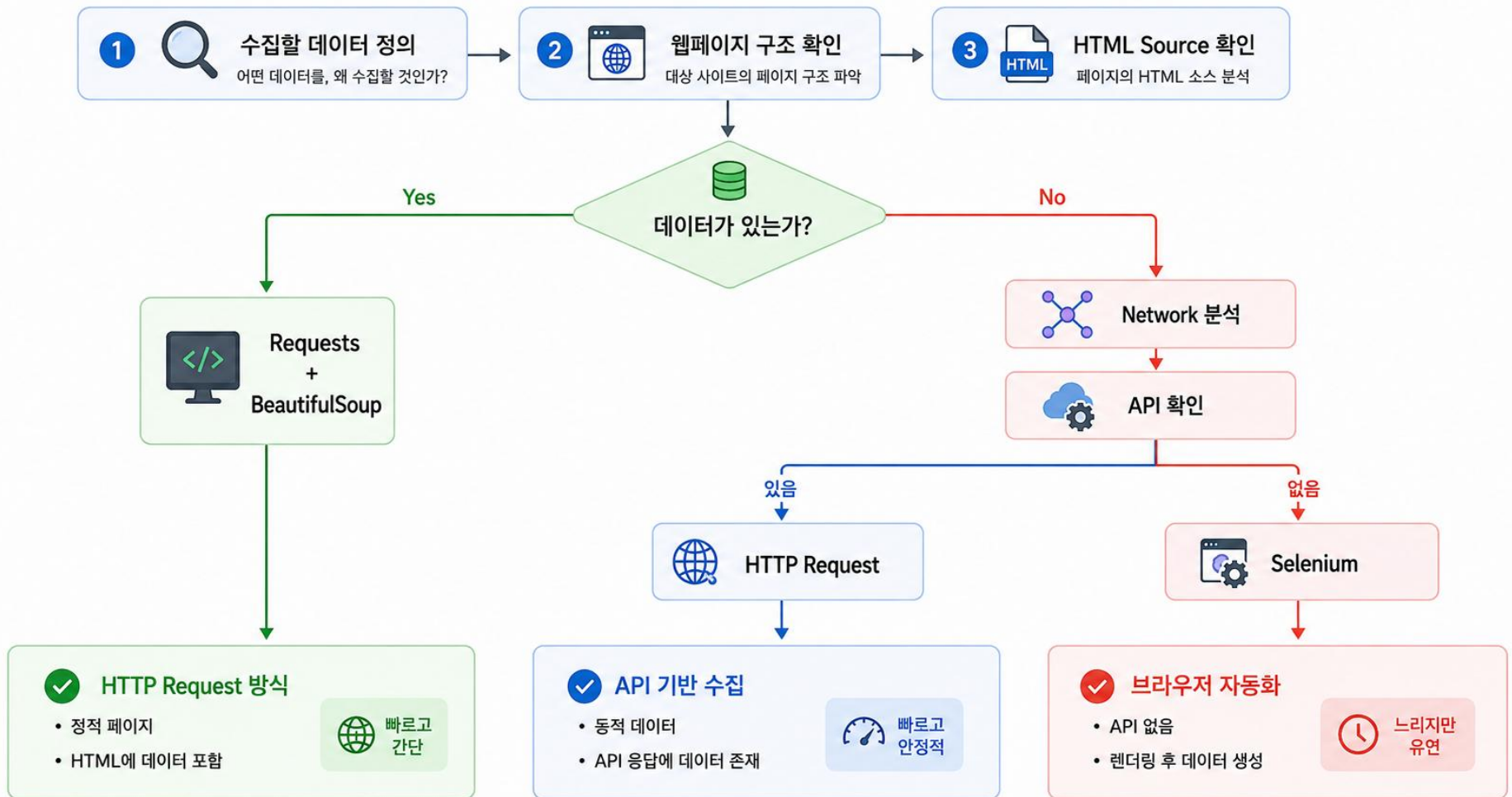
- ✓ 계층적 구조 (Tree)
- ✓ 부모-자식 관계
- ✓ 자바스크립트로 접근/조작 가능
- ✓ 브라우저가 HTML을 파싱하여 DOM 트리를 생성

개발자 도구 역할

- 크롤링에서 Chrome DevTools는 매우 중요한 분석 도구
- 실행
 - F12 또는 Ctrl + Shift + I
- 메뉴

메뉴	활용
Elements	HTML/DOM 구조 분석
Network	HTTP 요청/응답 분석
Console	JavaScript 및 DOM 확인
Application	Cookie/Storage 구조 확인

웹 데이터 수집 방법 판단 과정



문제

- 사용자가 Chrome에서 웹페이지를 요청했을 때 서버에 전달되는 메시지를 무엇이라고 하는가?
 - ① DOM
 - ② HTTP Request
 - ③ HTML Parser
 - ④ DataFrame

- HTTP Status Code 200의 의미는?
 - ① 접근 거부
 - ② 페이지 없음
 - ③ 정상 처리
 - ④ 서버 오류

문제

- HTML 문서를 분석하기 위해 사용하는 대표적인 Python 라이브러리는?
 - ① pandas
 - ② BeautifulSoup
 - ③ matplotlib
 - ④ numpy

- JavaScript 실행 후 나타나는 데이터를 브라우저 자동화를 통해 수집해야 하는 경우 적합한 도구는?
 - ① Selenium
 - ② NumPy
 - ③ OpenPyXL
 - ④ Matplotlib

문제

- 웹페이지 화면에는 데이터가 있지만 `requests.get()`으로 받은 HTML에는 데이터가 없다. 가장 먼저 무엇을 의심해야 하는가?
 - ① Python 설치 오류
 - ② JavaScript에 의한 동적 데이터 로딩
 - ③ CSV 인코딩 오류
 - ④ Pandas 버전 오류

웹 데이터 수집 종류

■ 뉴스 기사 크롤링

- RSS 서비스(예: <http://news.jtbc.joins.com/Etc/RssService.aspx>)
- XML 형식

■ SNS 데이터 크롤링

- 트위터, 댓글 등
- Text 형식 또는 JSON 형식

■ 웹 데이터 수집

- 웹 페이지 URL에서 원하는 정보를 수집
- HTML 형식

데이터를
선택적으로 가져옴

■ 공공데이터 수집

- Open API를 이용
- URL이 존재
- CSV 형식, 정형화된 데이터
 - Pandas 패키지의 read_csv("url")을 이용
- JSON 형식
 - Pandas 패키지의 read_json("url")을 이용

전체 데이터를
가져옴

학습 내용

- BeautifulSoup과 parser
- requests를 이용한 웹 데이터 수집
- Selenium을 이용한 웹 데이터 수집